Ejercicio #5

1. Análisis de Requisitos y Modelo de Clases (Patrón MVC)

Para cumplir con el requerimiento de aplicar el **patrón Modelo-Vista-Controlador (MVC)** y aprovechar al máximo la **herencia** y el **polimorfismo**, la arquitectura se divide en tres paquetes principales:

- **modelo**: Contiene la clase abstracta base Modulo y sus subclases (ModuloVuelo, ModuloTierra, ModuloEnergia), así como la clase Mision (o Simulador) que administra los recursos globales (energía disponible, datos en órbita y datos descargados) y la lista polimórfica de módulos.

- **vista**: Gestiona la interacción por consola con el operador (menús, despliegue de listas, resultados por ciclo y catálogos).

- **controlador**: Conecta la vista con el modelo, procesando la lógica del menú, las búsquedas, los ordenamientos y la ejecución de los ciclos de la simulación.

2. Jerarquía de Clases y Atributos (Herencia y Polimorfismo)

Todos los módulos comparten características comunes que se agrupan en una superclase abstracta.

Superclase Abstracta: Modulo

- **Atributos comunes**: String id, String nombre, double estadoSalud, boolean activo, double costoConstruccion.

- **Método abstracto clave**: public abstract void procesarCiclo(Mision mision);

- **Métodos comunes**: Getters, setters, sobreescritura de toString() y métodos de apoyo.

Subclases Especializadas

- **ModuloVuelo**:

    - *Atributos específicos*: String tipoInstrumento (cámara o sensor), double datosPorCiclo (MB), double consumoEnergiaCiclo, int ciclosAcumulados.

    - *Comportamiento polimórfico (procesarCiclo)*: Verifica si está activo y si la energía disponible en la misión es suficiente para cubrir su consumo. Si cumple, resta la

energía, acumula datos en órbita (a nivel de misión) e incrementa sus ciclos acumulados. Si no, informa el motivo sin realizar cambios.

- **ModuloTierra**:
    - *Atributos específicos*: double capacidadDescarga, double energiaNecesariaDescarga, double totalDescargado.
    - *Comportamiento polimórfico (procesarCiclo)*: Valda que esté activo, que haya energía suficiente en la misión y que existan datos pendientes en órbita. Descarga la cantidad permitida (sin superar lo disponible en órbita), consume la energía completa, actualiza los datos descargados globales y suma al acumulado del módulo.
- **ModuloEnergia**:
    - *Atributos específicos*: double energiaGeneradaCiclo, int ciclosAcumulados.
    - *Comportamiento polimórfico (procesarCiclo)*: Si está activo, produce una cantidad fija de energía que se suma inmediatamente a los recursos globales de la misión e incrementa sus ciclos acumulados.

3. Requisitos Funcionales del Sistema

El sistema implementa un menú interactivo en consola con las siguientes opciones operativas:

- **a) Listado general**: Muestra todos los módulos construidos en la misión con sus respectivas características y estados actuales.
- **b) Búsqueda específica**: Permite buscar un módulo ingresando su **ID** o su **nombre** exacto/parcial para desplegar toda su información detallada.
- **c) Catálogo por costo**: Despliega los módulos ordenados de forma ascendente o descendente según su **costo de construcción**, facilitando la gestión presupuestaria.
- **d) Avanzar un ciclo**: Ejecuta el método procesarCiclo() de cada módulo de manera secuencial según el orden establecido en la carga inicial. Los recursos de la misión (energía, órbita, descargados) se actualizan de manera inmediata tras la participación de cada componente.
- **e) Resumen de comunicaciones**: Genera un informe detallado que incluye:
    - Cantidad total de módulos de tierra y cuántos se encuentran activos.

- Capacidad total de descarga por ciclo de los módulos de tierra activos.
- Total histórico de datos descargados por todos los módulos de tierra.
- Identificación del módulo (o módulos) de tierra con mayor cantidad histórica de datos descargados.

4. Pautas para la Implementación en Java

1. **Carga Inicial**: En la clase controladora o en una clase de fábrica (DataInitializer), inicializa una única lista polimórfica (List<Modulo>) que contenga al menos **10 módulos combinados** (vuelo, tierra y energía).
2. **Principio de Responsabilidad Única (SRP)**: Asegúrate de que la vista solo imprima en consola y capture entradas del usuario, el modelo gestione el estado y las reglas de negocio, y el controlador coordine las acciones del menú.
3. **Buenas prácticas**: Incluye la sobreescritura correcta de los métodos toString(), encapsulamiento estricto con modificadores de visibilidad adecuados, documentación interna clara mediante comentarios y validaciones de errores de entrada en el menú.

Diagrama UML

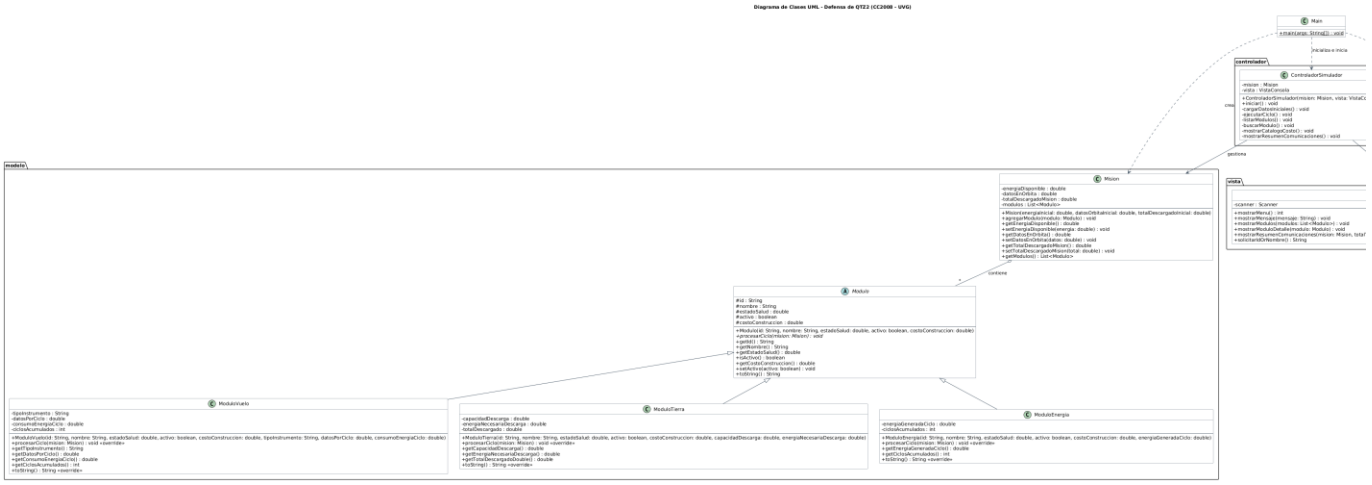


Diagrama de Clases UML - Defensa de QTZ2 (CC2008 - UVG)

